CrashGuard: Simulation-Based Evaluation of an IoT Crash Detection and Emergency Alert System for Two-Wheeler Riders

Adheesh Garg[1], Ayan Gatani[1], Aastha Gupta[1],
Syamasudha Veeragandham[2,*]

[1,2]School of Computer Science and Engineering,
Vellore Institute of Technology, Vellore, Tamil Nadu, India – 632014

adheesh.garg2023@vitstudent.ac.in
ayan.gatani2023@vitstudent.ac.in
aastha.gupta2023@vitstudent.ac.in
[*]Corresponding author: syamasudha.v@vit.ac.in

Abstract

We present CrashGuard, a low-cost handlebar device which detects a motorcycle crash and sends out an emergency alert, and we evaluate it by simulation alone. An ESP32 microcontroller reads a six-axis inertial sensor at 100 Hz, and the decision is taken by a small finite-state machine. For a crash to be confirmed four things must happen, in this order. The motorcycle must have been ridden; there must be an impact or a sudden unloading; a strong rotation must follow shortly afterwards; and the motorcycle must then lie tilted and still for two seconds. The device then turns on a light and a buzzer and gives the rider 15 seconds to cancel, after which a single event goes to a cloud service. We did not build a physical prototype. The results come from generated motion data, from a public motorcycle dataset and from the unchanged firmware, which we ran against simulated peripherals.

On a fresh set of 270 generated events CrashGuard rejected all 210 non-crash events and caught every one of the 30 high-side crashes. It missed, however, 17 of the 30 low-side crashes, which puts its overall sensitivity at 71.7% (F1 = 0.835). On the same events we also ran a rule built on the staged structure of the motorcycle fall detector of Boubezoul et al. (2013). This rule detected more crashes (86.7%) but gave 34 false alarms, and by McNemar's test the two rules differ significantly ($\chi^2 = 13.40$, $p = 0.00025$). The tilt, rotation and rest-hold thresholds were swept one parameter at a time. From the sweeps it appears that the 2 s rest hold is what suppresses the nuisance alarms, and that a lower rotation gate could win back some of the missed low-side crashes (the latter is still to be confirmed on separate data). Fourteen firmware tests exposed three defects. The rest timer keeps running when sensor reads are missing, a failed cloud connection holds up the main loop for 5 s, and an alert which falls due during a Wi-Fi outage is never retried. With a simple battery model the device lasts 12.64 hours at a 120 mA, 5 V load. Code, data and scripts are released, so every experiment reported here can be repeated.

Keywords: IoT; ESP32; MPU6050; Motorcycles; Detection; Simulation; Alerts; Safety.

1 Introduction

A motorcyclist who comes off the bike on an empty road can lie there unnoticed for quite some time. A device on the motorcycle which recognises the crash and calls for help would shorten this delay, but only if it does not raise false alarms so often that the rider switches it off. So the task is a detection problem with two kinds of error, and the two kinds do not cost the same. A missed crash may leave an injured rider without help. One false alarm costs little, but many of them make the device useless. In this paper we describe and evaluate CrashGuard, a prototype mounted on the handlebar and built around an ESP32 microcontroller and an MPU-6050 inertial sensor. We should say at once that no physical hardware was assembled. The evaluation relies on generated motion data, on public motorcycle recordings and on the real firmware run against simulated peripherals. In the rest of this section we go through the earlier work which shaped the design, one paper at a time.

Boubezoul et al. [1] were among the first to look at fall detection for powered two-wheelers (PTWs) as a problem of its own. Their main point is that a single threshold on acceleration is unreliable, as a motorcycle leans in ordinary riding and the road surface sends shocks through it. They detect a fall in stages; first the start of the fall, then the growth of the lean angle, and then a period of rest afterwards. CrashGuard keeps this staged view. Since it has to run on an inexpensive handlebar unit, the stages are written as a small finite-state machine, with a ride-qualification step put in front of them and a cancellation window behind. We compare our detector with a rule following the structure of their algorithm, and with two simpler rules, all on exactly the same six-axis data.

A few years later the same group published recordings of real PTW falls and critical manoeuvres [2]. Such data are rare, crashing a motorcycle on purpose being both expensive and dangerous. The falls serve as positive examples. The manoeuvres (hard braking, accelerating in a curve, a badly degraded track and so on) are negative examples, and they test whether a detector can keep quiet. We take four falls and seven manoeuvres from this set and use them for a limited purpose only, namely to check whether the signals cross the acceleration and rotation gates of CrashGuard. In other words we test the scale of our thresholds, not the full decision. All four falls cross both gates. The unit conversion, the filtering and the choice of time windows are published with the code.

Gelmini et al. [3] developed a crash detection algorithm for two-wheeled vehicles and tested it on experimental riding data. For us their work is mostly a lesson about coverage. Ordinary riding produces signals of many kinds, and a detector which has seen only a few convenient crash examples tells us little about how it will behave on the road. For this reason our simulated data contain braking, sharp turns, potholes, speed breakers, an impact followed by recovery and a parked motorcycle tipping over, besides two kinds of crash. Low-side and high-side crashes are kept as separate classes. This turns out to matter. In the fresh data every high-side crash is detected and all 17 misses are low-side crashes.

On the side of human falls, Sucerquia et al. [4] introduced SisFall, a dataset recorded with a device worn at the waist. Its signals come from a human body and not from a handlebar, so we do not use them. What we take from it is the design. It contains many everyday activities which are not falls, and a detector evaluated on it has to show that it stays quiet, not only that it fires. We follow the same principle; seven of our nine scenario classes are negative, and the class counts are given with every result. On the fresh events CrashGuard rejects all 210 negatives, though of course this holds only within our signal model.

Casilari et al. [5] compared the public datasets for wearable fall detection and found that they differ a good deal in sensor position, sampling rate, recorded activities and recording protocol. A sensitivity of, say, 95% measured on one dataset may tell little about another. The practical consequence for a new study is that the data have to be described well enough for a reader to judge what the numbers mean. We therefore report the channel units, the sampling rate, the random seeds, the noise and bias levels, the clipping limits and the kinematic transformation used by the generator. No event is ever split between the development, validation and test subsets, and we use a second random stream to check that the results repeat.

In a later study Casilari et al. [6] trained convolutional networks on several public fall datasets and showed that the outcome depends strongly on the repository used. This made us careful about what our own second dataset can show. A new random seed gives new events, but these come from the same generator, the same mounting position and the same model of the motorcycle. We call the result a fresh-seed evaluation, and claim nothing more for it. The public PTW recordings give evidence of another kind (about the physical scale of the signals), and the two are reported in separate tables. A stronger test, with whole vehicles or riding sessions held out, is left for future work.

Musci et al. [7] used recurrent neural networks (LSTMs) for online fall detection on an annotated version of SisFall. Their model reads a sequence of samples rather than one sample at a time, and we agree that most of the information lies in the order of events in time. We just encode it in a different way. In place of a learned model we have explicit timing rules, namely a correlation window between impact and rotation, a verification timeout and a hold time for the rest condition. One advantage of explicit rules is that the offline replay and the firmware run the same C++ code, so the two cannot disagree. Another is that when a fault is injected, the transition at which the behaviour went wrong can be read straight off the log.

Yu et al. [8] published KFall, a large dataset with temporal labels, meant for pre-impact fall detection. A pre-impact detector (for example one which triggers a wearable airbag) has to act within a fraction of a second. Our device acts after the event and has a few seconds to spare. The two problems thus need different timing measures, and it matters where a delay measurement starts and where it stops. We measure the time from the first candidate sample to confirmation separately from the 15 s cancellation window which comes after it. On the fresh events the median of the former is 2.33 s. The firmware tests check that the deadline at the end of the window is met to the millisecond.

Rodegast et al. [9] trained machine-learning models for motorcycle collision detection on simulated accidents and simulated normal riding. They aim at a passive safety system, which has to fire quickly and must not fire by mistake. We aim at a warning which the rider can cancel. A slower decision is then acceptable, but nuisance alarms are still costly. Like them we rely on simulation. Our data were generated after the detector rules had been fixed, and every rule we compare gets exactly the same events, so the trade-off between missed crashes and false alarms can be read directly from the confusion counts.

Leoni et al. [10] proposed an automatic eCall method for motorcycles based on vehicle dynamics. They separate high-side crashes, in which the vehicle rotates fast, from low-side crashes, in which the motorcycle slides out more slowly. This distinction is at the centre of our results. A threshold set for a fast and violent high-side crash may simply not see a slow slide. In our data CrashGuard rejects every non-crash event, but its low-side recall on the fresh set is only 43.3%, and we consider this the main weakness of the detector. It is reported separately, so that the overall accuracy of 93.7% is not taken to mean uniform protection.

Tian et al. [11] used a multi-head attention transformer for fall detection with a sensor worn on the shin. They report a test sensitivity of 90.5% and an overall accuracy of 93.1%, and discuss how class imbalance affects numbers of this kind. Their sensor sits on a human leg and ours on a handlebar. The figures are not comparable and we make no attempt to compare them. We cite the work as recent evidence of what learned sequence models can do. We have also taken from it the habit of reporting class counts, F1 and sensitivity next to accuracy; with 210 negatives and only 60 positives, accuracy on its own would flatter our detector.

Finally, Pradhan et al. [12] combined a dual-stream transformer with Kalman-filter fusion of inertial signals, and report an F1-score of 91.10% on SmartFallMM in a subject-wise evaluation. Two of their ideas seem worth adopting later on. One is the fusion of complementary inertial signals, the other is holding out whole groups during evaluation (for us these would be vehicles or riding sessions). At present CrashGuard uses three hand-designed features and fixed rules. Its contribution is elsewhere; it is a light decision sequence in which every state can be observed and tested.

1.1 Research Gaps and Demonstrated Improvements

Identified Research Gaps

Taken together, the work above leaves three gaps which matter for a small device of the kind we build.

Gap 1: Thresholds on acceleration alone. A fixed threshold on the acceleration magnitude cannot ignore hard road shocks (potholes, rumble strips, speed breakers) and at the same time detect slow, low-energy slides.

Gap 2: Overall accuracy hides missed low-side crashes. If most test events are negative, a high overall accuracy can go together with poor detection of one type of crash, typically the low-side slide with its gentle deceleration.

Gap 3: The alert chain is rarely tested. Most studies stop at classifier metrics. They do not test what the device does when the sensor fails, when the rider cancels, when the cloud service does not answer or when the battery runs down.

Demonstrated Engineering Contributions

The staged decision is due to Boubezoul et al. [1]. What we contribute concerns its use in a small handlebar device, and the way it is evaluated.

1. **Ride qualification.** A leaky-bucket filter of recent activity. A parked motorcycle which tips over, or a knock while standing, is then not taken for a crash.
2. **Local warning and cancellation.** A light and a buzzer, and after them a fixed 15 s window for the rider to cancel, before anything is sent.
3. **Comparison on identical data.** CrashGuard is compared on the same events with a rule following the structure of Boubezoul et al. [1] and with two simpler rules, using paired McNemar tests.
4. **Threshold sweeps.** The tilt, rotation and rest-hold thresholds are varied one at a time over 29 settings. The aim is to see which of them controls the missed low-side crashes and which the false alarms.
5. **Firmware fault tests.** The unchanged firmware is run against simulated peripherals in 14 scenarios. These exposed three defects, in the rest timer, in a blocking network call and in the missing retry after a network outage.

We repeat that nothing was built or measured in hardware. Every finding comes from simulation, from models of virtual peripherals or from public recordings.

1.2 Research Objectives

1. To implement and evaluate a 100 Hz six-axis state machine which detects crash-like events on a motorcycle handlebar, with a local warning which the rider can cancel.
2. To compare CrashGuard with a rule following Boubezoul et al. [1] and with two simpler rules, on the primary and on the fresh-seed event sets, and to report confusion counts, sensitivity, specificity, F1, delay and paired McNemar tests.
3. To find out, by sweeping the rest-tilt, rotation and rest-hold thresholds, how each of them affects low-side recall and false alarms.
4. To run the firmware with controlled virtual peripherals under sensor, calibration, timing and communication faults, and to record both the expected and the actual behaviour.
5. To publish the data, plots, circuit and architecture diagrams and the power model, along with a plain account of the limitations.

2 Related Work

Here the same sources are grouped by topic, and for each we say briefly what it does. How each of them affects our design was explained already in the introduction.

Motorcycle fall and crash detection. Boubezoul et al. [1] gave a simple fall detection algorithm for PTWs, which recognises a fall in stages from the motion of the vehicle. Later the same group released recordings of PTW falls and critical manoeuvres [2], and we use these for an external check of the signal scale. Gelmini et al. [3] studied crash detection for two-wheeled vehicles on experimental riding data, and stressed how varied ordinary riding is. Rodegast et al. [9] trained classifiers on simulated motorcycle collisions, for fast activation of passive safety systems. Leoni et al. [10] gave a dynamics-based eCall method which treats fast high-side motion and slower side slips differently.

Human fall detection and its datasets. SisFall [4] contains waist-worn recordings of falls and of activities of daily living. Casilari et al. [5] compared public datasets of this kind and showed how much they differ in sensor position, sampling and protocol; in a later study [6] they found that the results of convolutional networks depend on the dataset used. KFall [8] added synchronised temporal labels and benchmark algorithms for pre-impact detection.

Learned sequence models. Musci et al. [7] applied LSTM networks to online fall detection on an annotated version of SisFall. Tian et al. [11] used a multi-head attention transformer for a device mounted on the shin, and report 90.5% test sensitivity and 93.1% accuracy after class balancing and focal-loss training. Pradhan et al. [12] combined a dual-stream transformer with Kalman-based fusion and report 91.10% F1 in a 21-fold leave-one-subject-out evaluation on SmartFallMM. Mountings, labels and populations in these studies are not the same as ours, so their numbers serve as context and not as a ranking.

The cloud interface. The Blynk documentation on device templates and on event logging [13, 14] defines the device identity, the event code and the firmware call we use for reporting an incident. This is not detection research. It does, however, fix the boundary between the controller and the cloud, and our firmware tests record what happens at that boundary when an event is sent, when it is skipped and when the connection times out.

The sources are collected in Table 1. For each one the table gives the sensing domain or method, and says which part of our design or evaluation it informs. The rows describe how the sources relate to our work. They do not rank methods on a common dataset.

Table 1: Prior work on motorcycle crashes, human falls, datasets and the IoT interface, and the role of each in CrashGuard. The first column names the source, the second its sensing domain or method, and the third the design or evaluation decision it informs. The public motorcycle data are an external signal reference. The human-fall studies inform how we evaluate, but they use other mounting positions and other labels.

Source	**Sensing domain / method**	**Relevance to this work**
Boubezoul et al. (2013) [1]	Motorcycle fall algorithm	Origin of the staged decision. Our rules expose the candidate, rotation and rest states, and each of them can be replayed.
Boubezoul et al. (2019) [2]	Public PTW recordings	Four controlled falls and seven critical manoeuvres. We keep their labels and check the acceleration and rotation gates each on its own.
Gelmini et al. (2021) [3]	Motorcycle crash detection	Why we cover many vehicle behaviours; braking, turning, road shocks and recovery are explicit negative classes in our data.
SisFall (2017) [4]	Waist inertial dataset	Shows what everyday non-fall examples are worth. We carry over the principle but not the human-body labels.
Casilari et al. (2017) [5]	Dataset analysis	Why we report sensor location, sampling rate, units, event composition and the split procedure.
Musci et al. (2018) [7]	Online LSTM detection	A learned temporal model. CrashGuard expresses the same kind of timing as explicit correlation and rest rules.
Tian et al. (2026) [11]	Shin-mounted transformer	Recent attention-based work on human falls, sensitive to class balance. Mounted differently from our handlebar unit.
KFall (2021) [8]	Pre-impact timing dataset	Keeps pre-impact timing apart from post-event detection. We report the confirmation delay separately from the cancellation window.
Casilari et al. (2020) [6]	CNNs on several datasets	Why the fresh random stream is kept apart from the public PTW data, which come from another acquisition domain.
Rodegast et al. (2024) [9]	Simulated collisions, learned models	Fast activation of protective systems, whereas ours is a warning which can be cancelled.
Leoni et al. (2025) [10]	Dynamics-based motorcycle eCall	Separates fast high-side motion from slower side slips; this is why low-side misses are reported on their own.
Pradhan et al. (2026) [12]	Dual-stream inertial fusion	Kalman fusion with subject-wise evaluation. Suggests that we evaluate by vehicle or by session.
Blynk documentation [14]	Device event interface	Defines the event code and description. We instrument the submission, and do not treat it as equal to a notification received on a phone.

Summary of Gaps Addressed

In short, three gaps motivate the design as well as the experiments.

- **Choice of threshold.** A high acceleration threshold misses slow, low-energy falls, and a low one fires on ordinary unevenness of the road.
- **Scope of the evaluation.** For a safety device accuracy is not enough. Calibration, cancellation by the rider, alarm deadlines and the behaviour of the network all affect what happens after a real crash.

- **Openness.** If detector code, fixed event datasets and fault traces are not shared, other people cannot check the results or find the defects.

Problem Statement

We are given a 100 Hz stream of three-axis acceleration and three-axis angular rate, taken from a motorcycle handlebar. The task is to decide whether, during active riding, the stream contains an impact or an unloading, then a correlated rotation, and then a sustained tilted rest, in this order. Once this is confirmed the device turns on its local outputs and waits 15 seconds for a cancellation before it submits a single IoT event. The evaluation has to report false alarms, missed falls, detection delay and the behaviour under faults, through experiments which others can repeat.

3 Materials and Methods

3.1 Experimental platform and materials

The design targets an ESP32-WROOM-32 development board and a GY-521 MPU-6050 module, together with a red LED and its 220 Ω resistor, a 5 V passive piezo driven through a transistor, and an active-low push-button. Power is taken from a protected 2600 mAh 18650 cell through a TP4056/DW01A charger and protection module, followed by an MT3608 boost stage which gives 5 V. In the firmware the sensor is set to I^2C address `0x68`, a 400 kHz bus, the 44 Hz digital low-pass setting and 100 Hz output, with ranges of $\pm 16g$ for acceleration and $\pm 2000^\circ/s$ for angular rate [15]. As said in Section 1, no hardware was assembled or measured. The schematic, the interface calculations and the discharge curves are all results of design and simulation.

The experiments use the real detector header and the real firmware sketch [16]. A native C++ replay runs the detector on the generated motion data. In a separate harness the unchanged sketch is compiled together with deterministic stand-ins (test doubles) for the Arduino core, the Wire library, Wi-Fi and a Blynk-compatible service. Deadlines are driven by a virtual clock, and signed register values go in through the raw I^2C read path. The harness records the states of the digital pins, the tone frequency, the serial messages, the event payloads and the connection delays. Below, "the cloud" always means this simulated service.

Table 2 shows how each of the sensing, warning and input functions is wired to the controller. The harness watches exactly these pins, so the table links the circuit to the software as well.

Table 2: ESP32 interface allocation. GPIO stands for a general-purpose input/output pin, SDA and SCL are the I^2C data and clock lines, and ADC1 is the first analogue-to-digital converter. All logic shares a common ground. The outputs listed are the firmware commands seen in the harness. For the battery the entry gives the divider used to recover the cell voltage from the ADC reading.

Element	ESP32 interface	Configuration	Function
MPU-6050	GPIO21 / GPIO22	SDA / SCL	Acceleration and angular-rate acquisition.
LED	GPIO26	Active high	Visible crash warning.
Piezo driver	GPIO25	2400 Hz tone	Audible warning.
Cancel button	GPIO27	Internal pull-up	Cancel or acknowledge an alert.
Battery divider	GPIO34 / ADC1	100 kΩ / 100 kΩ	Cell voltage measurement.
Wi-Fi	Station interface	Phone hotspot	Blynk SSL event path.

3.2 System architecture

Figure 1 traces the data from the inertial sensor through the decision on the ESP32 to the local warning, and from there to the remote event. The power supply and the rider's button are drawn apart from this path. They support the decision but are not part of it.

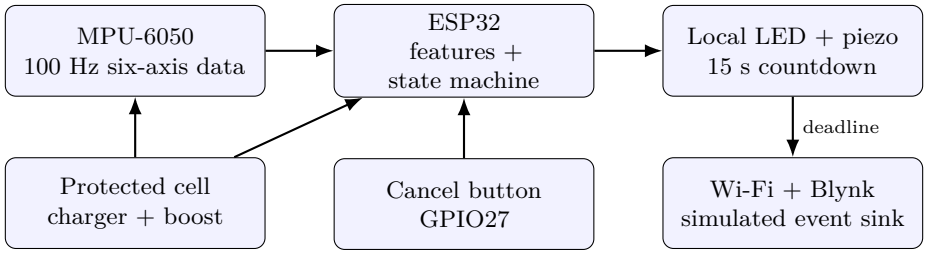


Figure 1: Architecture of CrashGuard: acquisition, local decision, cancellation by the rider and the remote event. In the harness an instrumented model of the service takes the place of the event sink.

3.3 Circuit diagram and virtual-hardware image

Figure 2 gives the circuit at connection level (power conditioning, the sensor, the battery measurement and the warning outputs), with labels marking the common nets. The component values are those used in the DC interface calculation below. The drawing also shows how the transistor drives the piezo.

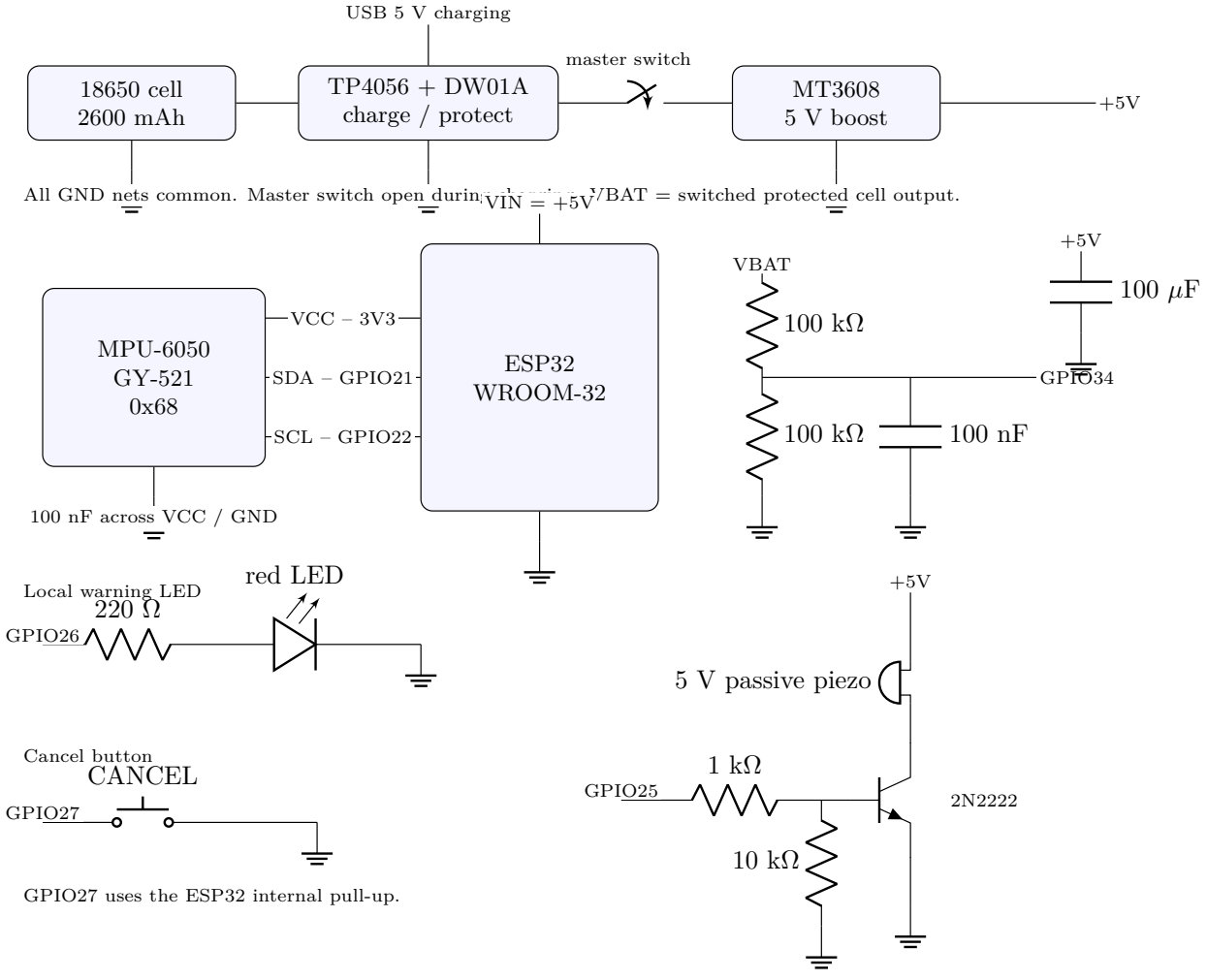


Figure 2: Connection-level circuit with power, sensing, battery divider, LED, transistor-driven piezo and cancel input. The master switch is open during charging, since the charger module does not share the load.

The GY-521 module works at 3.3 V, for power and for logic. GPIO25 drives the piezo through a 2N2222 transistor with a 1 kΩ base resistor and a 10 kΩ pull-down. A 100 nF capacitor goes from the battery divider to ground, and the 5 V rail is decoupled by 100 μF. We computed DC operating points for 39 combinations of cell voltage (3.0 to 4.2 V) and LED forward drop (1.8, 2.0 and 2.2 V). Over these the ADC input ranges

from 1.50 to 2.10 V, the divider draws 15 to 21 μA and the LED current lies between 5.0 and 6.82 mA. The values are calculated, not measured, and they do not depend on the battery experiment of Section 4.9.

Figure 3 is a capture of one executed state and not an arbitrary arrangement of parts. It was taken at the moment of confirmation, 3.610 s into the run. Its annotations give the LED level, the tone frequency, the countdown and the converted battery reading at that instant.

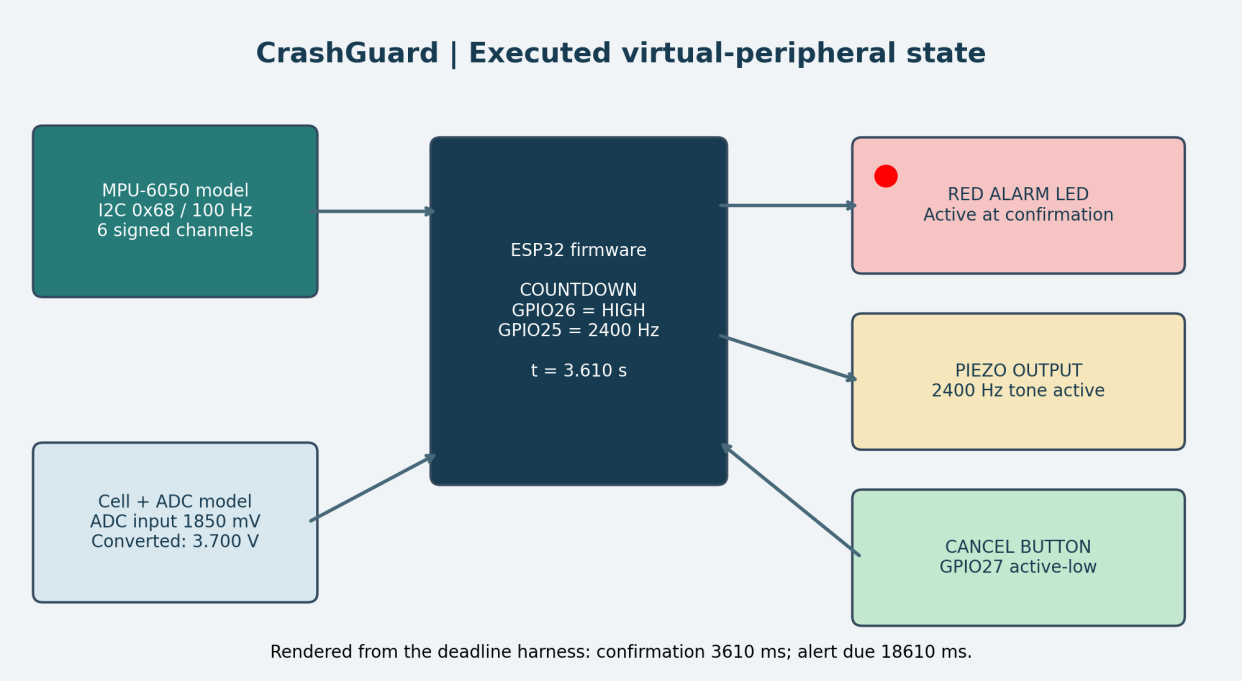


Figure 3: Virtual hardware at the confirmation instant, 3.610 s. Board and peripherals show the state recorded by the harness, i.e. LED on, 2400 Hz tone and the countdown running.

3.4 Dataset generation and preprocessing

The primary simulation consists of 675 events of ten seconds each at 100 Hz, which makes 675,000 six-axis samples. There are nine classes with 75 events each. Seven of them are negative (normal riding, potholes, speed breakers, hard braking, sharp turning, a parked tip-over and an impact followed by recovery), and the low-side and high-side crashes are positive. This gives 150 positive and 525 negative events. The generator is fixed by the seed 20260828. A second seed, 20260911, produces another 30 events per class, i.e. 270 fresh events and 270,000 samples.

Each event is put together from roll, pitch and yaw profiles, the gravity vector in the body frame, pulses for the manoeuvre or the impact, and noise, bias and clipping. In the body frame of the sensor gravity is

$$\hat{\mathbf{g}}_b = \begin{bmatrix} -\sin\theta \\ \sin\phi\cos\theta \\ \cos\phi\cos\theta \end{bmatrix} \in \mathbb{R}^{3\times 1}, \qquad \mathbf{g}_b = g_0\hat{\mathbf{g}}_b, \qquad \|\hat{\mathbf{g}}_b\|_2 = 1, \tag{1}$$

where ϕ and θ are the roll and pitch angles in radians, and the three entries follow the x, y and z axes of the sensor (subscript b for body frame). The hat marks a unit vector. Thus $\hat{\mathbf{g}}_b$ has no dimension and its entries read directly in units of g. Multiplying by standard gravity, $g_0 = 9.80665$ m/s^2, gives the physical vector $\mathbf{g}_b$ in m/s^2; note that g_0 is a constant and is not fitted. For a level sensor ($\phi = \theta = 0$) the vector is $[0, 0, 1]^\mathsf{T}$. Positive roll gives a positive y component, and positive pitch a negative x component. Yaw does not enter, as a rotation about the vertical leaves the direction of gravity unchanged.

Next, linear pulses for the road surface, the rider's inputs and the impacts are added to this gravity vector. For the gyroscope signals the generator does not simply differentiate the Euler angles. Instead it converts the angle rates $\dot{\boldsymbol{\Theta}} = [\dot{\phi}, \dot{\theta}, \dot{\psi}]^\mathsf{T}$ into body-frame angular velocities $\boldsymbol{\omega} = [\omega_x, \omega_y, \omega_z]^\mathsf{T}$ by the standard Tait–Bryan

Z-Y-X (yaw–pitch–roll) relation

$$\begin{bmatrix}\omega_x\\ \omega_y\\ \omega_z\end{bmatrix} = \mathbf{E}(\phi,\theta)\begin{bmatrix}\dot\phi\\ \dot\theta\\ \dot\psi\end{bmatrix} = \begin{bmatrix}1 & 0 & -\sin\theta\\ 0 & \cos\phi & \sin\phi\cos\theta\\ 0 & -\sin\phi & \cos\phi\cos\theta\end{bmatrix}\begin{bmatrix}\dot\phi\\ \dot\theta\\ \dot\psi\end{bmatrix}. \tag{2}$$

Here ψ is the yaw angle and $\mathbf{E}(\phi,\theta)$ is a 3×3 matrix depending only on roll and pitch. If the motorcycle rolls and yaws at once, as it does in a high-side crash, the matrix mixes the angle rates over the three gyroscope axes, and this is what a real body-fixed gyroscope would see. At zero roll and pitch the matrix is the identity and the body rates are just the angle rates.

The event time is drawn uniformly between 3.8 and 5.2 s. For acceleration the bias has standard deviation 0.018 g and the sample noise 0.022 g; for angular rate the corresponding values are 0.8 and 1.5 °/s. Samples are clipped to the full-scale ranges of the sensor. The final roll lies between 48 and 105° for low-side crashes and between 70 and 135° for high-side ones. Note that the low-side range goes below the 55° tilt gate of the detector on purpose, so some low-side crashes cannot be detected by design. Before vector addition and clipping the impact pulses are between 3.0 and 11.5 g for high-side crashes and between 0.8 and 5.5 g for low-side crashes.

For the external signal analysis we load seven critical manoeuvres and four falls from the published PTW supplement [2]. Only the finite numeric rows are kept. The six channels are filtered against aliasing and resampled from 1 kHz to 100 Hz (polyphase resampling with a Kaiser window); acceleration is converted from m/s^2 to g by dividing by 9.80665, and angular rate stays in degrees per second. The four fall windows are centred at 40.132, 34.288, 35.876 and 43.486 s and extend ±1.5 s each. The manoeuvres are used whole, as negative examples. Such short windows test single gates only. They are not a test of the full decision with rest and countdown.

3.5 Data split and evaluation protocol

Inside each scenario the primary events are split by the order of generation; the first 45 go to development, the next 15 to validation and the last 15 to test. This gives 405, 135 and 135 events (60, 20 and 20%). No event is divided between subsets, and the file `results/splits.csv` lists every assignment.

CrashGuard is a fixed set of rules without fitted weights. The split is therefore just a retrospective breakdown of one replay, and the test subset should not be read as a held-out test in the usual sense. The fresh set of 270 events was generated after the detector had been frozen for this study, and no threshold was chosen by looking at it. It is a second random stream from the same family of scenarios. The public PTW recordings are kept for the gate analysis, and the human-fall datasets serve only as context.

3.6 Features, calibration, and decision equations

At start-up the firmware averages 200 stationary samples. Calibration is repeated if the norm of the averaged acceleration falls outside 0.85–1.15 g, or if the RMS of the gyroscope is above 5 °/s. Otherwise the averaged acceleration is taken as the reference gravity vector and the three gyroscope means become bias offsets. Let $\mathbf{a} = [a_x, a_y, a_z]^\mathsf{T}$ be the acceleration vector and $\mathbf{r} = [r_x, r_y, r_z]^\mathsf{T}$ the reference gravity vector, both in units of g, and let $\boldsymbol{\omega} = [\omega_x, \omega_y, \omega_z]^\mathsf{T}$ be the angular rates in degrees per second. The detector works with three scalar features. The first one is the acceleration magnitude

$$A = \|\mathbf{a}\|_2 = \sqrt{a_x^2 + a_y^2 + a_z^2}, \tag{3}$$

where a_x, a_y and a_z are the components sampled at the same instant, divided by g_0. So $A = 1$ stands for one g, and the thresholds 2.0 and 0.35 used below mean $2g$ and $0.35g$. The firmware divides the raw signed counts by 2048 counts per g (the scale of the $\pm16g$ range), and a new sample comes every 10 ms.

The second feature is the magnitude of the angular rate,

$$\Omega = \|\boldsymbol{\omega}\|_2 = \sqrt{\omega_x^2 + \omega_y^2 + \omega_z^2}, \tag{4}$$

where ω_x, ω_y and ω_z are the bias-corrected rates about the three sensor axes, in degrees per second. Raw values are divided by 16.4 counts per degree per second (the ±2000°/s range), and then the calibration

means are subtracted. Because of the norm the gate does not care which axis carries the dominant rotation, nor what its sign is. The detector compares Ω with $90°/s$ in the search for a rotation and with $30°/s$ in the check for rest; these are thresholds and do not belong to the formula.

The third feature is the tilt, that is the angle between the present acceleration vector and the reference,

$$\Theta = \frac{180}{\pi} \arccos\left[\mathrm{clip}\left(\frac{\mathbf{a}^{\mathsf{T}}\mathbf{r}}{\|\mathbf{a}\|_2\|\mathbf{r}\|_2}, -1, 1\right)\right], \tag{5}$$

where $\mathbf{a}^{\mathsf{T}}\mathbf{r} = a_x r_x + a_y r_y + a_z r_z$ is the dot product, $\mathbf{r}$ is the mean of the 200 accepted calibration samples, and $\mathrm{clip}(u, -1, 1) = \min(1, \max(-1, u))$. Division by the two norms gives the cosine of the angle. The clipping guards arccos against round-off just outside its domain, and the factor $180/\pi$ turns radians into degrees, so that $\Theta \in [0, 180]$. When the product of the norms is below 10^{-6} the implementation returns zero tilt. As Θ is computed from the instantaneous acceleration, it gives the lean of the vehicle properly only while the motion is small. This is the case during the rest after a fall, and that is where we use it.

The decision then goes as follows. Riding activity is present if $|A - 1| \geq 0.08$ or $\Omega \geq 4°/s$. Activity evidence builds up to 1000 ms and decays at half that rate when there is no activity; as long as the evidence is full, a 5000 ms deadline for active riding is refreshed. Once riding is qualified, a candidate begins when $A \geq 2.0$ or $A \leq 0.35$. Within 750 ms of the candidate there must be a rotation with $\Omega \geq 90°/s$. After this the detector waits for the rest condition

$$R = (\Theta \geq 55°) \wedge (0.70 \leq A \leq 1.30) \wedge (\Omega \leq 30°/\mathrm{s}), \tag{6}$$

where $\wedge$ denotes logical AND. Thus R holds only when the motorcycle is strongly tilted, the acceleration is near $1g$ (within $\pm 0.30g$) and the vehicle hardly rotates. All comparisons include their limits. The predicate is instantaneous; the time requirements are imposed by the state machine and not by Equation (6).

A crash is confirmed once R has held for 2000 ms, and verification is given up 6000 ms after the candidate. Confirmation starts the alarm and the 15,000 ms cancellation window. The state machine needs only constant memory and does constant amount of work per sample.

3.7 Algorithm and flowchart

Table 3 lists the firmware procedure in the order in which it runs, from sensor initialisation to the single event submission, and shows where a failure of the sensor or the network changes the normal path.

Table 3: Algorithm 1: firmware procedure from sensor initialisation to the one-shot incident submission. Each row is one step of the implemented control path, together with its timing or failure branch. Samples are taken every 10 ms (100 Hz) and calibration uses 200 samples. If a sensor read fails the detector update is skipped. The deadline step records the outcome at the service, which does not imply that a notification reached anyone.

Step	Operation
1	Configure the GPIO pins and the MPU-6050; if the sensor does not answer, stop safely.
2	Collect 200 samples. If the device moved or the data are invalid, repeat until a valid reference is found.
3	Start Wi-Fi association in the background and set up the 10 ms sampling schedule.
4	Look for a press of the cancel button. It cancels a running countdown, or acknowledges an alert and silences the outputs.
5	At each sampling deadline read the 14-byte sensor burst. On failure log a warning and skip the detector update.
6	Compute A, Ω and Θ, then update the activity evidence and the candidate, rotation and rest states.
7	On confirmation set the LED high, start the 2400 Hz tone and note the start of the countdown.
8	After 15 s perform one event operation and record whether it was submitted, disabled, skipped for lack of Wi-Fi or skipped for lack of cloud.
9	Keep the local warning on until the rider acknowledges it, and go on sampling.

The flowchart of Figure 4 gives the same procedure as a chain of decisions with two return paths. A rejected candidate goes back to sampling, and a cancellation by the rider stops the warning before the event falls due.

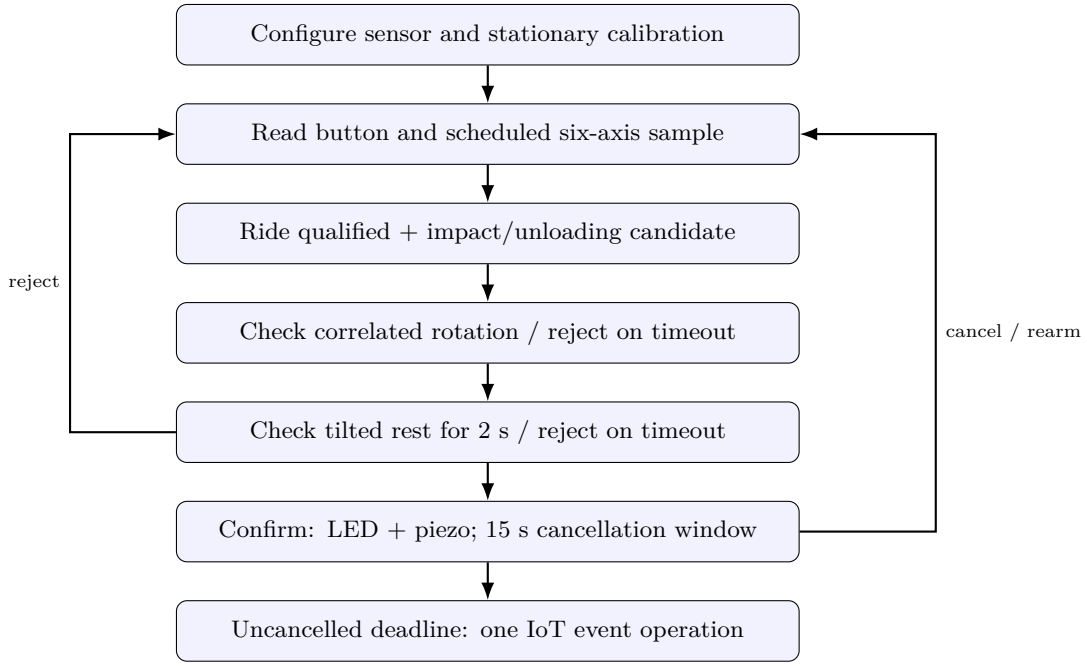


Figure 4: Flowchart of a successful detection, with the return paths for rejection and for cancellation. A failed sensor read also goes back to the sampling loop.

3.8 Baselines and metrics

We compare CrashGuard with three other rules. The *acceleration-only* rule fires as soon as $A \geq 2.0$. The *strict ordered* rule asks for $A \geq 4.0$, then $\Omega \geq 200°/\text{s}$ within 500 ms, and then a tilt of at least 60° with $0.70 \leq A \leq 1.30$ held for three seconds, all of it within eight seconds of the impact. The third rule follows the staged structure of the motorcycle fall detector of Boubezoul et al. [1], and we call it by their name. It has three stages. A fall starts when the roll rate $|\omega_x|$ reaches 60°/s, or when $A \geq 2.5$ or $A \leq 0.40$. The lean must then reach $\Theta \geq 55°$ within 2 s, or else the rule resets. Lastly the rest condition $\Theta \geq 55°$, $0.70 \leq A \leq 1.30$, $\Omega \leq 30°/\text{s}$ has to hold without a break for 1.5 s, and the attempt is given up 6 s after the start. We should be clear that these thresholds are our own choices, written to follow the structure of the published method. We do not claim that they are the exact values of the original paper, so the rule approximates their detector and is not a copy of it. Unlike CrashGuard it has no ride qualification.

Let TP, TN, FP and FN denote the numbers of true positives, true negatives, false positives and false negatives. They all count whole events, and in every experiment below the denominators of the metrics are positive. The first metric is accuracy,

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \tag{7}$$

the fraction of all events classified correctly. Its denominator is the size N of the evaluation set ($N = 675$ for the primary set, $N = 270$ for the fresh set). Negative events far outnumber positive ones, so accuracy can be high even when many crashes are missed. For this reason we never report it alone.

Sensitivity (also called recall, or the true-positive rate) is the fraction of crashes which are detected,

$$\text{Sensitivity} = \frac{TP}{TP + FN}. \tag{8}$$

The denominator is the number of labelled crashes, 150 in the primary set and 60 in the fresh one. This metric shows missed crashes directly, and we also give it separately for low-side and high-side crashes. For instance the fresh CrashGuard result is $43/(43 + 17) = 0.717$, or 71.7%.

Specificity is the fraction of non-crash events correctly rejected,

$$\text{Specificity} = \frac{TN}{TN + FP}, \tag{9}$$

with 525 negatives in the primary set and 210 in the fresh set. It describes nuisance alarms, but only within the negative events we chose to generate, and so it must be read together with how many such events there are and how varied they are. For the fresh CrashGuard result $TN = 210$ and $FP = 0$, and the observed value is 1. This is a finite sample. Rather than claim perfect rejection we give a Wilson interval for it below.

Precision is the fraction of detections which are real crashes,

$$\text{Precision} = \frac{TP}{TP + FP}. \tag{10}$$

Its denominator counts all events predicted positive, whatever their true class, so precision depends on how common crashes are in the test set. If a rule predicted no positive event at all the ratio would be undefined, and our code would return zero; this does not happen in any result reported here. For CrashGuard on the fresh set the precision is $43/(43 + 0) = 1$.

The F1-score combines precision and sensitivity,

$$F_1 = \frac{2TP}{2TP + FP + FN}, \tag{11}$$

that is, the harmonic mean of the two expressed through counts (the subscript 1 means that both are weighted equally). F1 is penalised by either kind of error. We report it as a decimal between 0 and 1, while accuracy, sensitivity and specificity are given in percent. For the fresh CrashGuard counts $F_1 = 86/103 = 0.835$.

The false-positive rate is

$$\text{FPR} = \frac{FP}{TN + FP}, \tag{12}$$

the fraction of non-crash events which raise an alarm, i.e. one minus the specificity. For the acceleration-only rule on the fresh set $FP = 108$ and $TN = 102$, so $\text{FPR} = 108/210 = 51.4\%$; for CrashGuard the numerator is zero. Note that this is a rate per event. A rate of false alarms per hour of riding would need riding time in the denominator, and our data do not give it.

The delay of a detection is the confirmation time minus the time at which the successful candidate began, and it is computed for detected crashes only. We briefly note that the acceleration-only rule has, by definition, no confirmation hold and hence zero delay. For a proportion $\hat{p} = x/n$ with x successes in n events we give 95% Wilson intervals [17],

$$\begin{aligned} p_{\mathrm{L}} &= \frac{\hat{p} + z^2/(2n) - z\sqrt{\hat{p}(1-\hat{p})/n + z^2/(4n^2)}}{1 + z^2/n}, \\ p_{\mathrm{U}} &= \frac{\hat{p} + z^2/(2n) + z\sqrt{\hat{p}(1-\hat{p})/n + z^2/(4n^2)}}{1 + z^2/n}, \end{aligned} \tag{13}$$

where p_{L} and p_{U} are the lower and upper limits, and $z = 1.959964$ is the 97.5th percentile of the standard normal distribution. For sensitivity $x = TP$ and $n = TP + FN$; for specificity $x = TN$ and $n = TN + FP$. The constants 2 and 4 come from the Wilson score construction. Unlike the simple normal interval, the Wilson interval remains informative when every observed event is a success, which is the case for our specificity. The intervals describe the uncertainty due to the finite number of events. Whether the generator resembles real riding is a separate matter, and on this they say nothing.

Finally, to test whether two rules differ on the same events we use McNemar's test with Edwards' continuity correction [18],

$$\chi^2 = \frac{(|b - c| - 1)^2}{b + c}, \tag{14}$$

where b is the number of events which CrashGuard gets right and the other rule gets wrong, and c is the number for which it is the other way round. Events on which the two rules agree play no part in the statistic. Under the null hypothesis that the two rules have equal error rates, χ^2 follows a chi-squared distribution with one degree of freedom. With few discordant events ($b + c < 25$) we use the exact two-sided binomial p-value instead, $p = 2\sum_{k=0}^{\min(b,c)} \binom{b+c}{k} 0.5^{b+c}$.

4 Experimental Results and Discussion

4.1 Experimental conditions

The C++17 programs are compiled with optimisation and strict warnings. Python with NumPy, SciPy and Matplotlib is used to generate the data, replay the detector, compute the metrics and draw the plots. Each event is processed by a freshly reset detector. The firmware harness has a virtual clock, a 14-byte register buffer, instrumented GPIO and tone outputs and a local Blynk-compatible sink. It runs the unchanged firmware and, on request, returns a failed probe, a truncated read, a lost link or a cloud timeout. No network credentials are needed.

We re-ran the pipeline on the 675 primary events and got the same confusion counts as before. Then we added the 270 fresh events, so that in total 945 events and 945,000 samples were processed. Besides these we ran 14 firmware scenarios, 39 DC operating points and nine battery-discharge cases. The experiments test different layers of the system and their counts are reported separately.

4.2 Original and fresh performance comparison

Table 4 compares the four rules on the 675 primary events, and from its confusion counts one can see where each score comes from. CrashGuard rejects far more non-crash events than the acceleration-only and the Boubezoul rules. The strict ordered rule also rejects every negative event, but it loses many more crashes.

Table 4: Event-level comparison of the four rules on the same 675 primary events (seed 20260828; 150 crashes and 525 non-crash events). TP, TN, FP and FN count events. Acc., Sens. and Spec. stand for accuracy, sensitivity and specificity in percent, and F1 is in $[0, 1]$. Each row sees identical input, so the differences are due to the rules alone.

Detector	TP	TN	FP	FN	Acc. (%)	Sens. (%)	Spec. (%)	F1
Acceleration-only	134	236	289	16	54.8	89.3	45.0	0.468
Strict ordered	39	525	0	111	83.6	26.0	100.0	0.413
Boubezoul et al. (2013)	135	436	89	15	84.6	90.0	83.0	0.722
CrashGuard (Ours)	114	525	0	36	94.7	76.0	100.0	0.864

Table 5 repeats the comparison on the 270 fresh events, with the rules unchanged. What we want to know is whether the pattern of the primary set survives on a new random stream. No threshold was chosen by looking at these results.

Table 5: The unchanged rules on 270 fresh events (seed 20260911; 60 crashes and 210 non-crash events). Columns are as in Table 4. This is a test of repeatability within the same generator, not a validation on other vehicles or road sessions.

Detector	TP	TN	FP	FN	Acc. (%)	Sens. (%)	Spec. (%)	F1
Acceleration-only	52	102	108	8	57.0	86.7	48.6	0.473
Strict ordered-threshold	12	210	0	48	82.2	20.0	100.0	0.333
Boubezoul et al. (2013)	52	176	34	8	84.4	86.7	83.8	0.712
CrashGuard (Ours)	43	210	0	17	93.7	71.7	100.0	0.835

On the primary set CrashGuard removes all 289 false alarms of the acceleration-only rule, and misses 20 more crashes than that rule does. Compared with the Boubezoul rule it removes 89 false alarms (specificity 100.0% against 83.0%) and misses 21 more crashes (sensitivity 76.0% against 90.0%). On the fresh set the pattern is the same. CrashGuard has none of the 34 false alarms of the Boubezoul rule, which lifts specificity from 83.8% to 100.0% and accuracy from 84.4% to 93.7% (F1 0.835 against 0.712), but it misses 9 more low-side crashes (71.7% against 86.7%). So neither rule dominates the other. CrashGuard pays for its silence on non-crash events with missed slow crashes. Whether this is the right trade depends on how riders react to repeated false alarms, and our data cannot answer that.

McNemar's test confirms that the errors of CrashGuard and of the Boubezoul rule are significantly different, with $\chi^2 = 40.81$ ($p = 1.68 \times 10^{-10}$) on the primary set and $\chi^2 = 13.40$ ($p = 0.00025$) on the fresh set. This needs to be read with some care, however. On the fresh set the discordant events are 34 non-crash events which only the Boubezoul rule gets wrong, and 9 crashes which only CrashGuard misses. The significance thus mostly reflects the false alarms of the Boubezoul rule, and it is no evidence that CrashGuard detects crashes better. Against the strict ordered rule CrashGuard detects significantly more crashes with no extra false alarm ($\chi^2 = 73.01$, $p < 10^{-15}$ on the primary set; $\chi^2 = 29.03$, $p = 7.1 \times 10^{-8}$ on the fresh set). Against the acceleration-only rule the difference is larger again ($\chi^2 = 232.44$, $p < 10^{-15}$ on the primary set; $\chi^2 = 82.09$, $p < 10^{-15}$ on the fresh set).

For the fresh set the 95% Wilson interval of sensitivity is 59.2–81.5%, and the observed specificity of 100.0% has the interval 98.2–100.0%. On the primary set the intervals are 68.6–82.1% for sensitivity and 99.3–100.0% for specificity. That no false alarm was seen in a finite set of negatives does not mean that none would occur under any riding condition.

The CrashGuard counts for each subset of the data are in Table 6. The table keeps the retrospective development, validation and test breakdown of the primary set apart from the fresh-seed experiment, as the two play different roles.

Table 6: CrashGuard confusion counts and F1 for each whole-event subset. Within each of the nine scenarios the primary simulation is split into 45 development, 15 validation and 15 test events (405/135/135 in total). The fresh-seed set adds 30 events per scenario. TP, TN, FP and FN sum to the number of events. The rules are fixed, so no weights are fitted in any subset.

Partition	Events	TP	TN	FP	FN	F1
Development	405	67	315	0	23	0.854
Validation	135	23	105	0	7	0.868
Retrospective test	135	24	105	0	6	0.889
Fresh-seed test	270	43	210	0	17	0.835

The F1 values of the three primary subsets are between 0.854 and 0.889, and the fresh value of 0.835 is slightly lower. With 30 positives or fewer in a subset, differences of this size are to be expected.

4.3 Scenario analysis and graphs

All seven negative classes are rejected completely in both sets, and no high-side crash is missed (75 of 75 are detected in the primary set and 30 of 30 in the fresh set). For low-side crashes the counts are 39 of 75 and 13 of 30. Hence each of the 36 primary misses and of the 17 fresh misses is a low-side crash. The low-side profiles have weaker impacts, slower motion and a smaller final lean than the high-side profiles, and the fixed gates accept only part of that distribution.

Figure 5 gives the fraction of correct decisions per class in both sets. What it shows is simple. Overall accuracy hides one class which fails often, the other eight being handled without a single error in these finite sets.

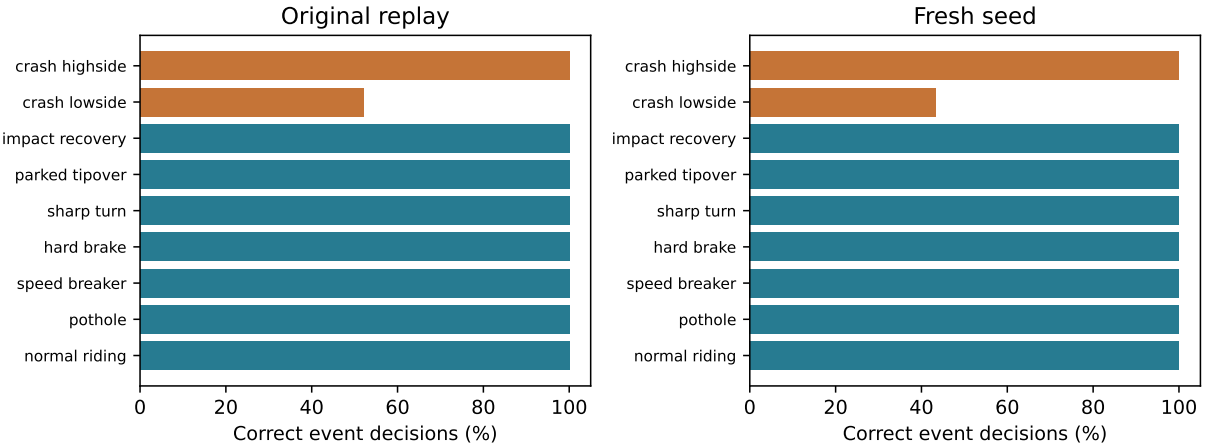


Figure 5: Fraction of correct event decisions per scenario, for the primary replay (left; 75 events per class) and for the fresh-seed set (right; 30 events per class). Crash classes are in orange and non-crash classes in blue.

Figure 6 plots the three features against time for one generated pothole, one parked tip-over and one low-side crash. The pothole gives a sharp acceleration peak above $3g$, but its rotation stays close to the $90°/s$ line and the tilt does not rise at all. The parked tip-over and the crash both rotate fast and end up strongly tilted. What separates them is the ride qualification, as the tip-over happens with no riding before it. This is why the same peak value need not lead to the same decision.

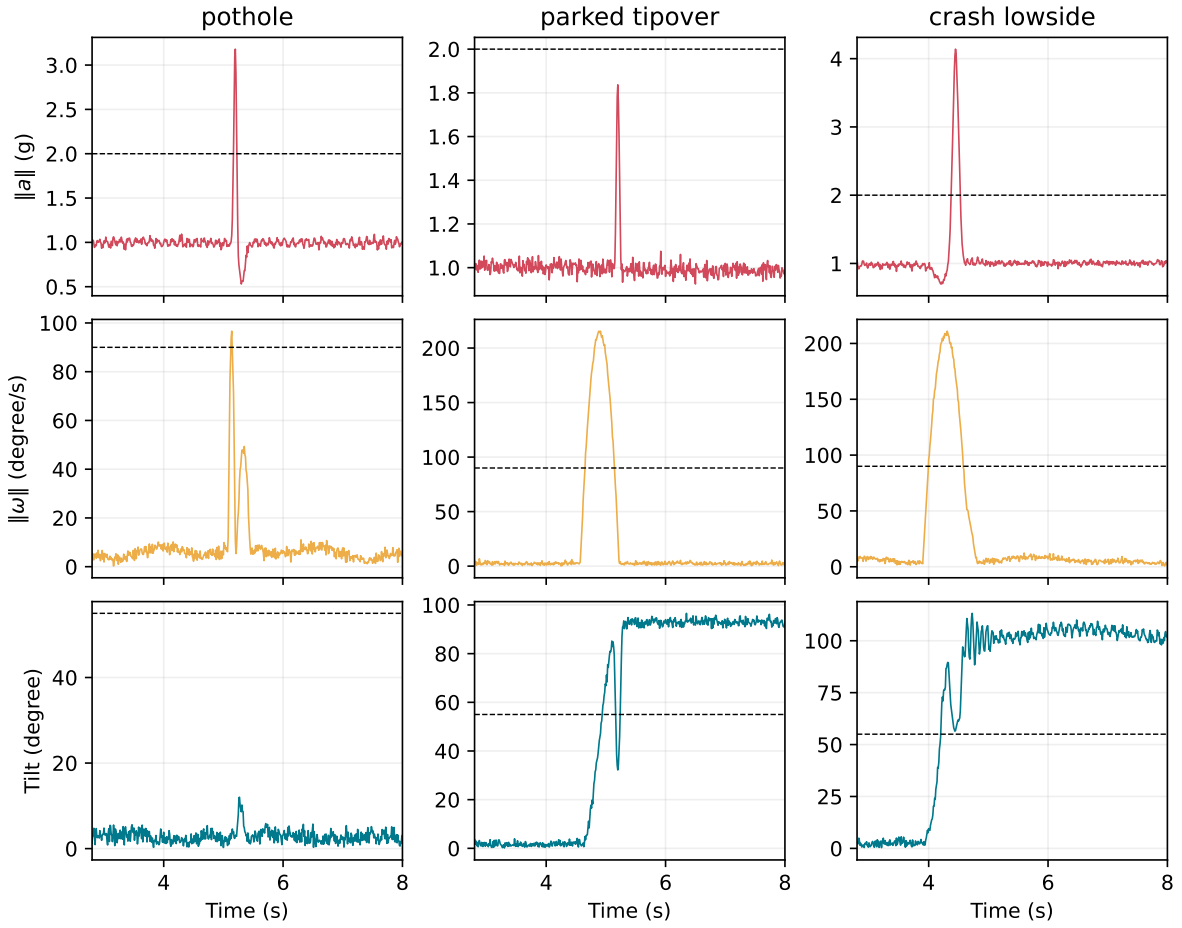


Figure 6: Acceleration magnitude $\|\mathbf{a}\|$ (top), angular-rate magnitude $\|\boldsymbol{\omega}\|$ (middle) and tilt (bottom) for a generated pothole, a parked tip-over and a low-side crash. The dashed lines mark the $2g$, $90°/s$ and $55°$ thresholds.

In Figure 7 the fresh confusion matrix is placed next to the distribution of confirmation delays. Note that the two panels have different denominators; the matrix counts all 270 events, the histogram only the 43 detected crashes. Most delays are between 2.2 and 2.4 s, just above the 2 s rest hold.

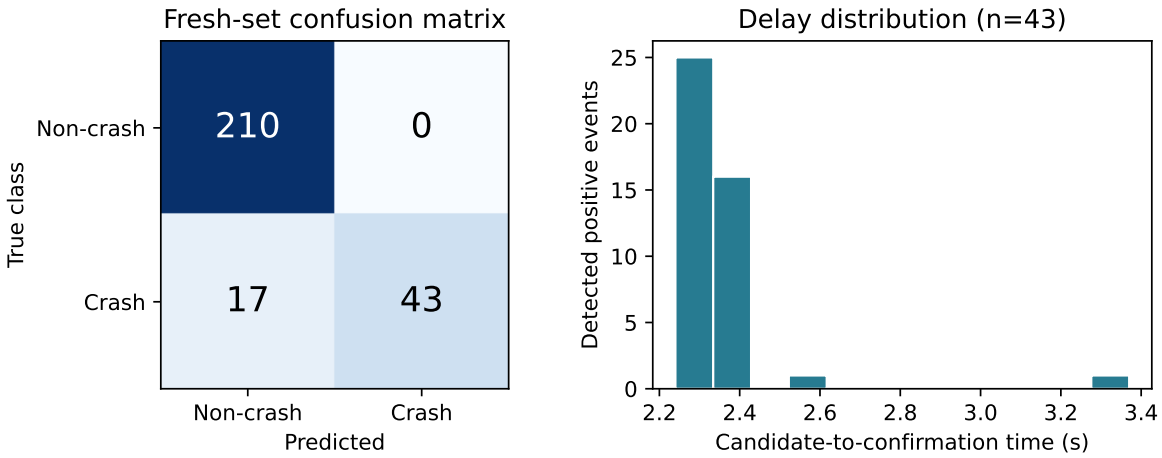


Figure 7: Confusion matrix of the fresh set (left) and the candidate-to-confirmation delays of the 43 detected crashes (right). Missed crashes appear in the matrix only.

4.4 External public-data gate analysis

Table 7 summarises the four public fall windows after unit conversion and resampling. From the peak and minimum acceleration we see whether the unloading gate opens, and from the peak angular rate whether the rotation gate opens. The last column records only that both gates are crossed at some point in the window. It does not say that the full decision would be made.

Table 7: Extremes of the inertial signals in four public PTW fall windows, after anti-aliased resampling from 1 kHz to 100 Hz. Each row covers 3 s centred on the event. Peak and minimum A are in g, and peak Ω in degrees per second. "Both gates" means that the acceleration condition ($A \geq 2.0$ or $A \leq 0.35$) and the rotation condition ($\Omega \geq 90°/s$) are each met somewhere in the window. It says nothing about their timing, nor about the rest condition.

Fall recording	**Peak A (g)**	**Min. A (g)**	**Peak Ω (°/s)**	**Both gates**
Slippery straight road	1.169	0.013	179.4	Yes
Leaning motorcycle	1.878	0.041	144.1	Yes
Roundabout	1.690	0.011	150.5	Yes
Curve	1.436	0.130	151.6	Yes

All four falls open both CrashGuard gates. In every case it is the unloading branch ($A \leq 0.35$) which opens, as no peak reaches $2g$. None of the falls reaches the $4g$ and 200°/s gates of the strict rule, and this explains part of its poor recall. On the other hand, three of the seven manoeuvres (accelerating in a curve, accelerating on a straight line and hard braking) also open both CrashGuard gates somewhere in their recordings. The gates by themselves do not, therefore, separate falls from manoeuvres. That is left to the requirements on timing, riding and rest, which we cannot check on these recordings since the fall windows are short. The full features of all eleven recordings are in `external_ptw_features.csv`.

4.5 Timing, cancellation, and IoT communication

From candidate to confirmation the median delay is 2315 ms on the primary set and 2330 ms on the fresh set. The strict ordered rule needs 3120 and 3130 ms, the Boubezoul rule 2280 and 2325 ms. Most of the delay is thus due to the rest hold itself. Adding the intentional 15 s cancellation window, the time from the candidate to the event deadline at the fresh median is 17.33 s, before any delay in connecting to the service.

In the harness riding is qualified first. A rotation is then injected at 1500 ms and an impact at 1600 ms. The tilted rest starts at 1610 ms, confirmation comes at 3610 ms and the event falls due at 18,610 ms. The test checks that the state is still `COUNTDOWN` at 18,600 ms and has become `ALERT_DUE` ten milliseconds later. After confirmation GPIO26 is high and the tone is at 2400 Hz. The simulated service receives exactly one `crash_confirmed` event, with a description 143 bytes long.

In another test the cancel button on GPIO27 is pressed at 4010 ms. The outputs switch off and the detector stays idle until the run ends at 20,000 ms. Four outcomes are distinguished on the event path: submission to the service, a disabled configuration, a Wi-Fi failure and a cloud failure. The event description gives the acceleration, angular rate and tilt at confirmation, and says that no location fix is available (the device has no GNSS receiver). Event identity and payload follow the Blynk firmware event interface [14].

Taken together, the virtual-hardware image and the state traces work as a replay dashboard, in which the sensor values, the countdown, the alarm outputs and the event outcome all come from one run. A deployed application could show the same fields through a Blynk device template. Continuous telemetry on virtual pins would be an extension. In the sketch evaluated here the only remote operation is the event submission.

4.6 Threshold sensitivity and calibration analysis

To see how much the results depend on the chosen thresholds we varied one parameter at a time, with the others at their configured values, and replayed the 675 primary events for each setting. Parameters and ranges were as follows.

1. The rest tilt threshold Θ_{tilt}; 13 values from 40° to 70° in steps of 2.5°.
2. The rotation gate Ω_{rot}; 10 values from 60 to 150°/s in steps of 10°/s.
3. The rest hold T_{hold}; 6 values from 1.0 to 3.5 s in steps of 0.5 s.

In all this makes 29 settings. The sweep reuses the events on which the configuration was developed, so it describes how the rules behave and is not an independent test. Figure 8 shows the results.

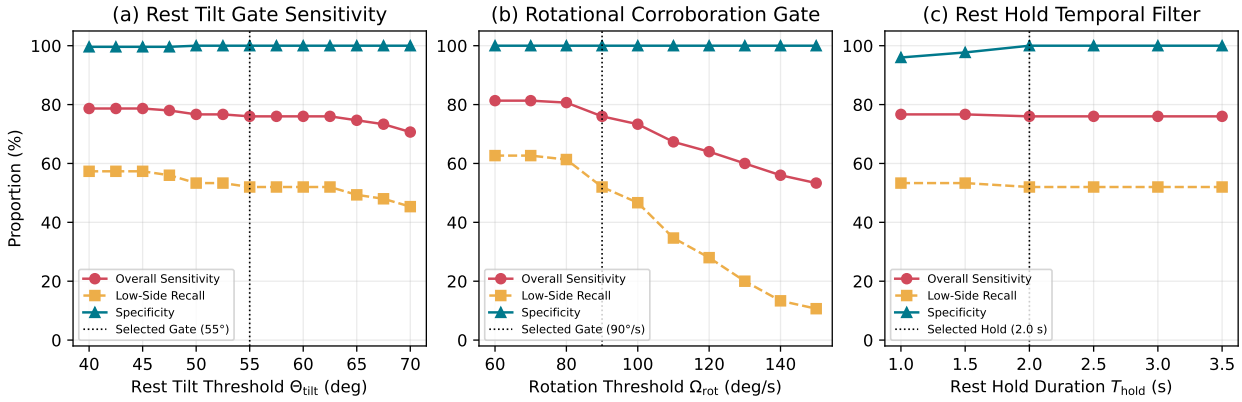


Figure 8: One-at-a-time threshold sweeps on the 675 primary events: (a) rest tilt threshold, (b) rotation gate, (c) rest hold. Each panel gives overall sensitivity, low-side recall and specificity, and the dotted line marks the configured value (55°, 90°/s, 2.0 s).

Tilt. For every tilt threshold down to 50° specificity stays at 100%. From 47.5° down, two non-crash events are accepted (99.6%). Inside the range free of false alarms the tilt threshold matters surprisingly little; sensitivity is 76.7% at 50–52.5° and 76.0% from 55° to 62.5°, a difference of one event. Going down to 45° wins back four more low-side crashes (low-side recall 57.3%, overall 78.7%), at the price of the two false alarms. Above 62.5° sensitivity drops, and at 67.5° and 70° even high-side crashes start to be lost (98.7% and 96.0%); at 70° the overall sensitivity is 70.7%. Our conclusion is that the tilt gate is not the main cause of the low-side misses.

Rotation. Over the whole range from 60 to 150°/s specificity stays at 100%, while low-side recall depends strongly on the gate. It is 62.7% at 60 and 70°/s, 52.0% at the configured 90°/s and just 10.7% at 150°/s. On these data, then, a rotation gate of 70°/s would detect eight more low-side crashes (overall sensitivity 81.3% in place of 76.0%) without one false alarm. This is perhaps the most useful outcome of the sweep, as it shows that the configured point is not the best of those free of false alarms. We have kept the configuration

as it is, however. The observation comes from the development data, and the rejection of negatives may rest largely on the other conditions; before a lower rotation gate is adopted it has to be checked on held-out events.

Rest hold. Here it is the other way round. Sensitivity hardly changes (76.7% at 1.0 and 1.5 s, 76.0% from 2.0 s on), but specificity falls to 97.7% (12 false alarms) at 1.5 s and to 96.0% (21 false alarms) at 1.0 s. The sweep does not record which classes these false alarms come from. A likely source is the impact-with-recovery class, whose generated lean lasts 0.7 to 2.1 s, but we have not verified this. Longer holds give no gain in specificity, and since the hold is part of the confirmation delay, each extra half second of hold adds about half a second of delay by construction. Of the holds tried, 2 s is thus the shortest one which removes all false alarms.

4.7 Firmware implementation verification and code defect discovery

Table 8 pairs each injected condition with the intended response and with the trace actually observed. Most cases check that the firmware behaves as specified. Three of them (T9, T11 and T13) isolate defects in this particular code. We stress that these are defects of our prototype implementation and not properties of IoT hardware in general; the harness found them before any board had been flashed.

Table 8: Fourteen experiments with the exact firmware and deterministic models of the sensor, GPIO, clock and service. ID names the case, Stimulus is the injected input or failure, Expected the required behaviour, and Actual the observed trace with its interpretation. Times are in virtual-clock milliseconds unless written in seconds. Alarm 1/0 means that the local warning is commanded on/off. For T9, T11 and T13 a passing assertion reproduces a known weakness, and does not certify tolerance to faults.

ID	Stimulus	Expected behaviour / criterion	Actual result and interpretation
T1	Sensor absent at boot	Probe failure; safe stop; alarm off	Fatal marker and safe-stop loop observed; alarm 0. Conforms.
T2	Ten-byte sensor burst	Reject incomplete 14-byte read	Warning emitted; state remains IDLE. Conforms.
T3	I^2C error status	Skip invalid sample	Warning emitted; state remains IDLE. Conforms.
T4	100°/s during calibration	Reject motion above 5°/s RMS	Calibration rejected. Conforms.
T5	Stationary gravity input	Accept 200 valid samples	Calibration accepted after 200 samples. Conforms.
T6	ADC returns 1850 mV	Reconstruct cell voltage with factor two	3.700 V returned. Conforms.
T7	Cancel at 4010 ms	Silence outputs and suppress deadline	Alarm 0; IDLE through 20 s. Conforms.
T8	Uncancelled crash	15,000 ms from confirmation to one event	3610 ms confirmation; 18,610 ms deadline; one event. Conforms.
T9	Missing reads during rest	Desired: require continuous valid rest evidence	Confirmation at 3610 ms despite a sample gap. Continuity defect exposed.
T10	Wi-Fi unavailable at deadline	Explicit skip; local alarm kept	Zero sink events; Wi-Fi skip marker; alarm 1. Conforms.
T11	Cloud connection fails	Explicit failure; characterise blocking	Zero sink events; 5000 ms blocking; alarm 1. Blocking risk exposed.
T12	Connected simulated sink	Correct event code and bounded payload	One event; code `crash_confirmed`; 143 bytes. Conforms.
T13	Wi-Fi recovers after deadline	Desired: eventual incident delivery	Zero events after recovery. Missing retry exposed.
T14	Build without credentials	Disabled marker; local alarm kept	Not-configured marker; alarm 1. Conforms.

All fourteen assertions pass. For T9, T11 and T13, passing means that the known defect is reproduced. We go through the three defects in turn.

T9: rest-timer continuity. The rest timer measures the time elapsed between timestamps, without checking that samples actually arrived in between. If the I^2C reads fail during the rest, time still counts. In the test a valid rest sample at 1610 ms is followed by bus failures at 2000 and 3000 ms and then by a valid sample at 3610 ms. The crash is confirmed, although the sensor was silent for 2000 ms. The fix is to restart the rest timer whenever the gap between valid samples is longer than, say, 50 ms.

T11: blocking connection. The cloud connection call is synchronous, with a 5000 ms timeout. If the service cannot be reached the call holds the main loop for those 5 s, and sampling deadlines are missed. Production firmware needs a non-blocking network state machine.

T13: no retry. An incident which falls due while Wi-Fi is down is skipped, and the state moves on to `ALERT_COMPLETED`. Nothing is sent when Wi-Fi comes back. The event must be stored and retried.

The GPIO and tone assertions are about the commanded outputs only. The present firmware gets no feedback from its actuators, so it cannot detect a disconnected piezo or a dead LED. Checking that the outputs physically work would need extra sensing, which we leave for future work.

4.8 Expected performance versus actual performance

Table 9 puts each design objective next to the experiment which tests it, and says what follows. Classification, timing, recovery and endurance appear side by side, the objectives which were not met included.

Table 9: Objectives, observations and interpretation. The classification rows give correct decisions or false alarms over the relevant number of events in the primary and fresh sets. The timing rows use the virtual clock of the firmware, and the recovery rows say whether continuous rest evidence and eventual delivery are achieved. The endurance row is for the 120 mA, 5 V, 90% efficiency case. Meeting one row does not mean that the others are met.

Objective	**Actual experiment**	**Interpretation**
Reject road disturbances	0/525 primary and 0/210 fresh false alarms	Achieved in both scenario sets.
Detect crash-like events	114/150 primary; 43/60 fresh	Low-side sensitivity is the main weakness.
Allow cancellation	Alarm silenced at 4010 ms; IDLE through 20 s	Cancellation conforms.
Enforce 15 s alert interval	18,610 − 3610 = 15,000 ms	Exact deadline on the virtual clock.
Keep valid rest evidence	Sensor gap spans the rest timer	Timer must restart after a gap.
Recover event delivery	Zero events after the link returns	Persistent retry needed.
Run for 12 h in the nominal case	12.64 h at 120 mA and 90% efficiency	Met in the nominal case; heavier loads fall short.

4.9 Power simulation

The battery is modelled as a reservoir of usable charge which is drawn down step by step. The operating time comes out of the model; it is not put into it. The initial charge is

$$q_0 = C_{\mathrm{nom}} f_{\mathrm{use}} = (2.6\,\mathrm{Ah})(0.90) = 2.34\,\mathrm{Ah}, \tag{15}$$

where $C_{\mathrm{nom}} = 2.6$ Ah is the nominal capacity of the cell, $f_{\mathrm{use}} = 0.90$ is the fraction of it we treat as usable, and the subscript 0 marks the start of discharge. Both inputs are fixed parameters of the simulation.

While the cell discharges, its voltage falls linearly with the remaining charge,

$$V(q) = V_{\min} + (V_{\max} - V_{\min})\frac{q}{q_0} = 3.0\,\mathrm{V} + (1.2\,\mathrm{V})\frac{q}{q_0}, \tag{16}$$

where q is the remaining usable charge in Ah, with $0 \le q \le q_0$, so that q/q_0 is the state of charge. The voltage is $V_{\min} = 3.0$ V for an empty cell and $V_{\max} = 4.2$ V for a full one. The model is simple on purpose; a real lithium-ion cell has a flatter curve in the middle.

The boost converter keeps its output at a fixed voltage, and so the current taken from the cell rises as the cell voltage drops. At integration step k,

$$I_{\mathrm{cell},k} = \frac{V_{\mathrm{out}} I_L}{\eta V(q_k)}, \tag{17}$$

where q_k is the remaining charge, $V_{\mathrm{out}} = 5.0$ V is the output voltage, I_L is the load current (90, 120 or 180 mA) and η is the converter efficiency (0.80, 0.90 or 0.95). The numerator is the output power in watts; dividing it by the efficiency and the cell voltage gives the cell current in amperes. We simulated all nine combinations of load and efficiency.

The charge is then updated by a forward Euler step,

$$q_{k+1} = q_k - \frac{I_{\mathrm{cell},k}\,\Delta t_s}{\kappa_t}, \qquad \Delta t_s = 1\,\mathrm{s}, \quad \kappa_t = 3600\,\mathrm{s\,h^{-1}}, \tag{18}$$

where Δt_s is the time step and κ_t converts ampere-seconds into ampere-hours. Iteration stops after the first step at which the remaining charge is no longer positive. The last step may therefore overshoot zero, by less than one step's worth of charge.

In the nominal case ($I_L = 0.120$ A and $\eta = 0.90$) the cell is empty after 45,490 steps of one second, i.e. after 12.64 h. Over the nine cases the runtime goes from 7.49 to 17.78 h. These runtimes are plotted in Figure 9; each point is one complete integration, and along each curve the efficiency is fixed while the load varies.

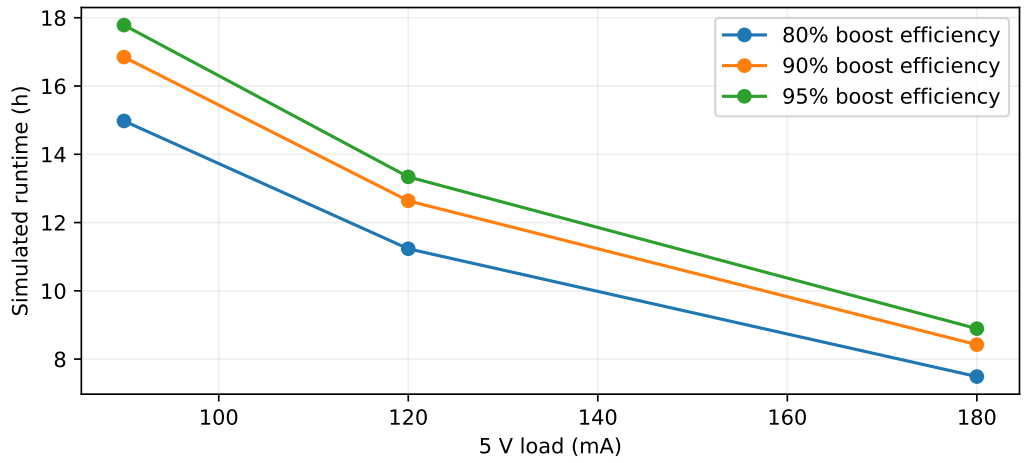


Figure 9: Simulated runtime for the nine combinations of 5 V load (90, 120, 180 mA) and boost efficiency (80, 90, 95%). Both the falling cell voltage and the converter loss are in the model.

The numerical model can be checked against a closed-form integral. With the same linear voltage profile and constant load and efficiency, the runtime in hours is

$$t_{\mathrm{lin}} = \frac{\eta q_0 (V_{\mathrm{min}} + V_{\mathrm{max}})}{2 V_{\mathrm{out}} I_L}, \tag{19}$$

with all symbols as before, and $(V_{\mathrm{min}} + V_{\mathrm{max}})/2 = 3.6$ V the mean cell voltage. The units reduce from Ah·V/(V·A) to hours. In the nominal case the formula gives 12.636 h, which matches the numerical result to within the one-second step.

For comparison, if the cell voltage is held constant at $V_{\mathrm{const}} = 3.7$ V and the other values are unchanged, the runtime $t_{\mathrm{const}} = \eta q_0 V_{\mathrm{const}} / (V_{\mathrm{out}} I_L)$ is 12.99 h. The difference of 0.35 h comes only from using 3.7 V in place of the mean of 3.6 V. Neither calculation accounts for ageing of the cell, for temperature, or for the voltage sag under short bursts of load such as Wi-Fi transmission.

4.10 Performance comparison with research literature

Table 10 has two parts. In Part A the four rules are run on the same 270 fresh events, so its rows can be compared directly. Part B lists published results from studies of human falls. These studies use other sensor

positions (the shin, the waist), other motions and sometimes other goals, for instance triggering an airbag before impact. Their numbers are given for context, and cannot be ranked against Part A.

Table 10: Comparison by domain. Part A: the four rules on the identical 270 fresh simulated PTW events (seed 20260911; 60 crashes, 210 non-crash events). Part B: published results of studies on human motion, with other protocols. Numbers can be compared only inside Part A.

Method	Evaluation domain	Reported outcome	Relationship
Part A: motorcycle fall detection on identical fresh simulated events ($N = 270$)			
CrashGuard (Ours)	Fresh simulated PTW	Sens. 71.7%; Spec. 100.0%; Acc. 93.7%; 0 FP; F1 0.835	Staged state machine with ride qualification and 15 s cancellation.
Boubezoul et al. [1]	Same fresh events	Sens. 86.7%; Spec. 83.8%; Acc. 84.4%; 34 FP; F1 0.712	Rule following the published PTW algorithm; recall higher, specificity lower ($\chi^2 = 13.40$, $p = 0.00025$).
Acceleration-only	Same fresh events	Sens. 86.7%; Spec. 48.6%; Acc. 57.0%; 108 FP; F1 0.473	High recall but many alarms on road shocks ($\chi^2 = 82.09$, $p < 10^{-15}$).
Strict ordered	Same fresh events	Sens. 20.0%; Spec. 100.0%; Acc. 82.2%; 0 FP; F1 0.333	No false alarms, but 80% of crashes missed ($\chi^2 = 29.03$, $p = 7.1 \times 10^{-8}$).
Part B: literature from other domains (other sensors, mountings and objectives)			
Tian et al. (2026) [11]	Shin-mounted, human falls	Test sens. 90.5%; Acc. 93.1%; F1 0.916	Attention transformer for wearable fall detection; mounting and mechanics differ.
KFall deep model [8]	Falls of older adults	Sens. 99.32%; Spec. 99.01%	Pre-impact detection on waist-worn data; timing and dynamics differ.

Part A shows the trade-off which runs through the whole paper. CrashGuard gives up sensitivity on low-energy falls (71.7% against 86.7% for the Boubezoul rule) and in return has no false alarms at all (specificity 100.0% against 83.8%). Riders being what they are, it seems plausible that a device which often raises false alarms would be switched off, and if so the trade is worth making. We have no data on how riders behave, though, and the question should be settled by a field study rather than by argument.

5 Limitations and Future Work

The main weakness of the detector is its low-side recall. CrashGuard missed 17 of the 30 fresh low-side crashes (43.3% recall), and these make up all of its missed crashes in both sets. In a low-side crash the motorcycle slides down without a large impact, and the slower slides go unseen by the fixed gates. The sweep in Figure 8 suggests where to begin. With a lower rotation gate eight low-side crashes were recovered in the development data, with no false alarms, and this has to be confirmed on held-out events. Beyond that one could track the lean angle adaptively, use the vehicle speed from wheel sensors or GNSS, or train a small sequence model for slow slides.

The generator uses proper rigid-body kinematics (Equation (2)), but a good deal of the physics of a real motorcycle is not in it. The travel of the front suspension, the flexibility of the frame, the nonlinear friction between tyre and road, engine vibration and resonance of the mounting bracket are all missing. Eventually the detector has to be tested in instrumented crash tests and on long road recordings, over different surfaces and with different mounting positions. Note also that the low-side generator itself puts some final roll angles below the 55° gate. Part of the low-side miss rate is therefore a property of our data and part of it a property of the detector, and we have not separated the two.

The comparison with Boubezoul et al. rests on our own choice of thresholds for their staged method (Section 3.8). Another, equally reasonable, choice might shift their sensitivity and specificity, and what Table 5 says about that rule should be read with this in mind.

The firmware tests revealed three defects, all of which must be fixed.

1. *Rest-timer continuity (T9).* The rest timer must restart each time the gap between valid samples goes beyond a freshness bound, say 50 ms.
2. *Blocking network call (T11).* The synchronous 5000 ms connection call has to be replaced by a non-blocking network state machine, so that sampling is not held up.
3. *Event persistence (T13).* An incident falling due during a network outage must be stored in non-volatile memory and retried, with back-off and deduplication, once the link returns.

Two further limitations are more basic. The device cannot tell the emergency services where the rider is, and it has never been built. A low-power GNSS receiver, or a Bluetooth link to the rider's phone, would give the location. Before anything can be said about the device outside simulation a real board is needed, tested on a vibration bench and sealed against water and dust (IP67, say).

6 Conclusion

We have described CrashGuard, a crash detection and emergency alert prototype for powered two-wheelers, and evaluated it in simulation. A 100 Hz six-axis state machine decides whether there has been a crash. A light and a buzzer then warn the rider, who has 15 seconds to cancel before an event is sent to the cloud.

On 270 fresh simulated events CrashGuard detected all 30 high-side crashes and rejected all 210 non-crash events, but it missed 17 of the 30 low-side crashes. Its overall sensitivity is thus 71.7%, with 100.0% specificity, 93.7% accuracy and F1 = 0.835. For the 675 primary events the corresponding figures are 76.0% sensitivity, 100.0% specificity and F1 = 0.864.

On the same fresh events a rule following the staged method of Boubezoul et al. (2013) detected more crashes (86.7%) but raised 34 false alarms (83.8% specificity; F1 0.712), and by McNemar's test the errors of the two rules differ significantly ($\chi^2 = 13.40$, $p = 0.00025$). The one-at-a-time sweeps show three things. The 2 s rest hold is what removes the false alarms; within the range free of false alarms the tilt threshold has little effect; and on the development data a rotation gate of 70°/s in place of 90°/s would detect eight more low-side crashes, again without false alarms. This last point is the obvious next experiment.

Running the unchanged firmware against simulated peripherals was useful in itself. Before any hardware existed it found three defects, namely the rest timer running across missing reads, a blocking cloud connection and the missing retry after a network outage. The battery model gives 12.64 hours at the nominal 120 mA, 5 V load.

We have tried to report the weaknesses of the system as plainly as its strengths, and to make every number in the paper reproducible from the released code. We hope this makes CrashGuard a useful starting point for others who build low-cost safety devices for riders of two-wheelers.

CRediT authorship contribution statement

- **Adheesh Garg**: Conceptualization, Methodology, Software, Validation, Investigation, Data curation, Visualization, Writing – original draft.
- **Ayan Gatani**: Conceptualization, Methodology, Software, Validation, Investigation, Data curation, Visualization, Writing – original draft.
- **Aastha Gupta**: Conceptualization, Methodology, Software, Validation, Investigation, Data curation, Visualization, Writing – original draft.
- **Syamasudha Veeragandham**: Supervision, Writing – review and editing.

Data Availability

All material is on GitHub at `https://github.com/qwerty-dvorak/crashguard-iot/tree/main/paper`. The directory holds the manuscript in PDF and LaTeX, the bibliography, the ESP32 firmware, the circuit

definitions, the simulation and evaluation scripts, the generated datasets, the per-event predictions, the data partitions, the experiment logs, the figures and the results in machine-readable form, and every path and command resolves inside it. The third-party PTW recordings are due to Boubezoul et al. [2]; the source URL and the SHA-256 checksum of their supplementary archive are given in the data documentation. Running `sh reproduce.sh` regenerates the motion data, replays the detectors, runs the firmware and power simulations and rebuilds the manuscript. The procedure is described in the README of the repository.

References

[1] A. Boubezoul, S. Espié, B. Larnaudie, and S. Bouaziz, "A simple fall detection algorithm for powered two wheelers," *Control Engineering Practice*, vol. 21, no. 3, pp. 286–297, 2013.

[2] A. Boubezoul, F. Dufour, S. Bouaziz, B. Larnaudie, and S. Espié, "Dataset on powered two wheelers fall and critical events detection," *Data in Brief*, vol. 23, p. 103828, 2019.

[3] S. Gelmini, S. C. Strada, M. Tanelli, S. M. Savaresi, and C. D. Tommasi, "A novel crash detection algorithm for two-wheeled vehicles," *IEEE Transactions on Intelligent Vehicles*, vol. 6, no. 1, pp. 88–99, 2021.

[4] A. Sucerquia, J. López, and J. Vargas-Bonilla, "Sisfall: A fall and movement dataset," *Sensors*, 2017.

[5] E. Casilari, J.-A. Santoyo-Ramón, and J.-M. Cano-García, "Analysis of public datasets for wearable fall detection systems," *Sensors*, 2017.

[6] E. Casilari, R. Lora-Rivera, and F. García-Lagos, "A study on the application of convolutional neural networks to fall detection evaluated with multiple public datasets," *Sensors*, 2020.

[7] M. Musci, D. D. Martini, N. Blago, T. Facchinetti, and M. Piastra, "Online fall detection using recurrent neural networks," 2018. [Online]. Available: https://arxiv.org/abs/1804.04976

[8] X. Yu, J. Jang, and S. Xiong, "A large-scale open motion dataset (kfall) and benchmark algorithms for detecting pre-impact fall of the elderly using wearable inertial sensors," *Frontiers in Aging Neuroscience*, 2021.

[9] P. Rodegast, S. Maier, J. Kneifl, and J. Fehr, "On using machine learning algorithms for motorcycle collision detection," 2024. [Online]. Available: https://arxiv.org/abs/2403.09491

[10] J. Leoni, S. Gelmini, G. Panzani, M. Tanelli, and S. M. Savaresi, "Automatic ecall in powered two-wheelers: A dynamics-based approach," *IEEE Transactions on Intelligent Transportation Systems*, vol. 26, pp. 4365–4379, 2025. [Online]. Available: https://doi.org/10.1109/tits.2025.3545907

[11] J. Tian, S. Xu, P. Gerstoft, M. Sarkar, A. A. Moore, P. Mercier, and C. Paolini, "A multi-head attention transformer model for wearable in situ fall detection," *IEEE Access*, vol. 14, pp. 85 242–85 262, 2026.

[12] A. Pradhan, S. Alamgeer, R. Suvvari, S. T. Haque, and A. H. H. Ngu, "Dual-stream transformer with kalman-based sensor fusion for wearable fall detection," *Big Data and Cognitive Computing*, vol. 10, p. 90, 2026. [Online]. Available: https://doi.org/10.3390/bdcc10030090

[13] Blynk, "Device template documentation," 2026, accessed 11 September 2026. [Online]. Available: https://docs.blynk.io/en/concepts/device-template

[14] ——, "Log event: Firmware api," 2026, accessed 11 September 2026. [Online]. Available: https://docs.blynk.io/en/blynk-library-firmware-api/log-event

[15] TDK InvenSense, *MPU-6000 and MPU-6050 Product Specification, Revision 3.4*, 2013. [Online]. Available: https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet.pdf

[16] CrashGuard project, "Shared detector, firmware, dataset generator and firmware experiment harness," 2026, reproducible source distributed with this report; see Data Availability and the repository README.

[17] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[18] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.